



Universidad Nacional Autónoma de México
Facultad de Ciencias

Ingeniería de Software
Microservicio de Adogta

Versión del Documento: 1.0

Profesora:

Karla Socorro García Alcántara

Ayudantes:

- Maitreyi Martínez Jiménez
- Luis Gerardo Bernabe Gomez

Equipo:

Kernel Crew y La Sonora Dinamita.

Integrantes:

- Carrillo Gonzalez Luis Israel
- Flores Galeana Daniela
- González Castillo Patricio Salvador
- Guadalupe Abrego Karla Sheridam
- Raya Garcia Marco Antonio
- Rubio Resendiz Marco Antonio

Índice

Introducción.	3
Propósito del documento.	3
Alcance del sistema.	3
Actores del sistema para esta versión.	3
Actores internos.	3
Tecnologías Utilizadas.	3
Arquitectura General.	4
Requerimientos funcionales.	4
RF1: Procesamiento y Traducción LLM. El sistema debe recibir un nombre de raza y una	5
RF2: Mapeo de nivel de sociabilidad con mascotas, sociabilidad con niños, nivel de energía e independencia. El sistema debe asignar un valor numérico entero del 1 al 5 (donde 1 es muy bajo y 5 es muy alto) basado en el análisis del texto del temperamento ingresado.	5
RF3: Manejo de errores de validación (Cliente). Si el microservicio recibe peticiones con	5
RF4: Manejo de errores del Servicio LLM (Servidor). Si ocurre una falla en la	5
Casos de uso.	5
CU1: Clasificar temperamento de raza y traducir a español.	5
Restricciones técnicas de la iteración	6

Introducción.

Propósito del documento.

El presente documento detalla la arquitectura, el alcance, los requerimientos y el diseño de Microservicio para Adogta, el cual forma parte integral del sistema de adopción. Este microservicio tiene como responsabilidad exclusiva procesar descripciones de razas y su temperamento, apoyándose en Inteligencia Artificial mediante modelos de lenguaje (LLM) a través de la API de OpenAI.

El propósito de este documento es describir formalmente las especificaciones de diseño y operación del microservicio. Está dirigido al equipo de desarrollo y servirá como guía para su mantenimiento, correcta integración con el backend principal y manejo adecuado de errores y latencias inherentes al uso de IA.

Alcance del sistema.

El microservicio expone un único endpoint (/clasificar-temperamento) diseñado para recibir el nombre y una descripción textual del temperamento de un animal (comúnmente proveniente de The Dog API o The Cat API). Su función se limita a:

- Traducir el nombre de la raza y el temperamento al idioma español.
- Evaluar y clasificar el nivel de energía del animal mapeando a una escala numérica del 1 al 5.
- Devolver una respuesta estructurada o notificar si ocurre algún error durante el proceso de
- Validación o inferencia del LLM.

Actores del sistema para esta versión.

Actores internos.

- **Backend de Adogta:** Es el encargado de enviar la solicitud (request) con los textos en inglés y de interpretar la respuesta JSON. El microservicio no está expuesto directamente al frontend ni a usuarios finales.

Tecnologías Utilizadas.

A continuación se describen las tecnologías empleadas en la implementación del microservicio en la segunda iteración.

Backend

Categoría	Tecnología	Justificación
Lenguaje y Framework	Python 3 / FastAPI	Permite un desarrollo ágil de APIs asíncronas de alto rendimiento y proporciona documentación automática Swagger.
LLM / IA	LangChain y OpenAI	Se utilizan para orquestar los prompts y asegurar, con <code>with_structured_output</code> , que la respuesta siga estrictamente los esquemas definidos.
Validación de Datos	Pydantic	Garantiza que la estructura de los datos de entrada (texto, nombre) y salida (traducciones, nivel 1-5) sean consistentes antes de procesarlos o retornarlos.
Servidor Web	Uvicorn	Servidor ASGI ligero y ultrarrápido para ejecutar la aplicación FastAPI de manera local o en despliegue.

Arquitectura General.

La arquitectura del microservicio sigue un modelo por capas o módulos bien delimitados para garantizar la escalabilidad y facilitar pruebas:

- Routes (app/api/routes.py): Es el punto de entrada. Define el endpoint POST /clasificar-temperamento. Recibe la petición HTTP y delega la ejecución al servicio.
- Service (app/services/llm_service.py): Contiene toda la lógica de negocio y la conexión con el modelo de OpenAI. Utiliza un ChatPromptTemplate para inyectar el contexto de experto en comportamiento canino.
- Models/Schemas (app/models/schemas.py): Define los esquemas ClasificacionRequest (nombre, texto_a_clasificar) y ResultadoClasificacion (nombre en español, temperamento en español, nivelEnergia).
- Configuración (app/core/config.py): Se encarga de cargar y validar las variables de entorno, aislando credenciales sensibles (API Key) del resto del código.

Requerimientos funcionales.

Los siguientes requerimientos funcionales corresponden a las funcionalidades implementadas en la Iteración 4.

RF1: Procesamiento y Traducción LLM. El sistema debe recibir un nombre de raza y una

descripción de temperamento en formato JSON, procesarlos utilizando un modelo LLM y retornar dichos valores traducidos íntegramente al español.

RF2: Mapeo de nivel de sociabilidad con mascotas, sociabilidad con niños, nivel de energía e independencia. El sistema debe asignar un valor numérico entero del 1 al 5 (donde 1 es muy bajo y 5 es muy alto) basado en el análisis del texto del temperamento ingresado.

RF3: Manejo de errores de validación (Cliente). Si el microservicio recibe peticiones con

formato inválido, campos vacíos o tipos de datos incorrectos, debe retornar de forma inmediata un error HTTP 500 al backend.

RF4: Manejo de errores del Servicio LLM (Servidor). Si ocurre una falla en la comunicación con la API de OpenAI, el sistema debe capturar la excepción y devolver un mensaje de error estructurado con código HTTP 500 (Internal Server Error) al backend, indicando que el servicio de clasificación no está disponible en ese momento.

Casos de uso.

CU1: Clasificar temperamento de raza y traducir a español.

Actores: Backend de Adogta.

Descripción: El backend envía los datos obtenidos de The Dog API (nombre y temperamento en inglés de una raza) al microservicio para su respectiva traducción al español y cálculo del nivel de sociabilidad con niños, sociabilidad con otras mascotas, nivel de energía e independencia en un rango de 1 a 5.

Precondiciones:

- El backend de Spring Boot ha recolectado de forma exitosa los datos del animal.
- El microservicio se encuentra en ejecución con una API Key de OpenAI válida y operativa.

Flujo principal:

1. El backend emite una petición POST al endpoint /clasificar-temperamento.
2. El microservicio valida la petición utilizando Pydantic.
3. El servicio LLM inyecta los datos dentro del prompt establecido y hace la llamada a OpenAI.

4. El modelo clasifica y devuelve los resultados (nombre en español, temperamento en español, parámetros necesarios con su mapeo a un valor entre 1 y 5).
5. El microservicio responde con un código HTTP 200 y el JSON estructurado al backend.

Flujo alternativo:

- Si ocurre un problema de comunicación con la API de OpenAI, el microservicio intercepta el fallo y responde con un código HTTP 500, adjuntando un mensaje de error. El backend principal deberá procesar este error (por ejemplo, mostrando un mensaje genérico).

Restricciones técnicas de la iteración

- Dependencia de API Key de paga: El sistema depende de una clave de API activa de OpenAI para funcionar. Si los créditos se agotan, o la suscripción no es válida, las llamadas fallarán.
- Latencia esperada por la consulta al LLM: Dado que se realiza una petición de red a un modelo generativo grande, la latencia es sustancialmente mayor que la de un endpoint común. El backend de Spring Boot debe manejar tiempos de espera (timeouts) prolongados o implementar un procesamiento asíncrono para no bloquear los hilos principales ni empeorar la experiencia del usuario final.